Introduction to HTML and CSS?

→HTML (Hyper text markup language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a webpage using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multi-media content. HTML documents are interpreted by web browsers to display the content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage.

→CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment,

dimensions, and layouts. CSS also supports responsive design, allowing webpages to adapt to different screen sizes and devices. CSS can be applied using inline, internal, or external stylesheets.

→ HTML and CSS work together to develop webpages, where HTML provides the structure and content, while CSS provides the styling, layout, visual presentation. The combination of both forms the foundation of modern web development and enables the creation of organized, attractive, and user-friendly web interfaces.

Difference between MERN Stack and Full Stack

→ MERN Stack is a Javascript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js, MongoDB is used for database management, Express.js handles the backend framework and API's, React.js is used to develop fronted user interface, and Node.js

provides the server side runtime environment

→ Full stack development refers to the development of both the frontend and backend of a web application along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

8/07/2026

HTML components:

1. HTML Tags:- Defines the structure and elements of a webpage.
2. Headings - used to display titles and headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag.
4. Hyper Links - used to navigate between webpages using <a> tag.
5. Images - used to display images using <image> tag.
6. Lists - used to create ordered and unordered lists using <ol>, <ul>
7. Tables - used to organize data in rows and columns.
8. forms - used to collect user input using elements such as text boxes, buttons and checkboxes.
9. DIV and span - used to group and

organize webpage content.

10. Semantic Elements - Elements such as <header>, <nav>, <selection>, <article> <footer>, provide meaningful webpage structure.

CSS components:

1. Selectors - Select HTML elements to apply styles.
2. properties - Define the visual characteristics of elements.
3. Values - specify the setting of a CSS property.
4. Colors - Define texts, background, border colors.
5. Fonts - control font family, size, style and weight.
6. Backgrounds - set background colors and images.
7. Borders - Define the border style, width and radius.
8. Margin - controls the space Outside an element.
9. padding - controls the space inside element.
10. Layout - controls the arrangement of elements using flexbox, and positioning.

HTML vs HTML5

HTML	HTML5
• Older version of HTML.	• Latest version of HTML.
• Limited multimedia support.	• Supports audio and video.
• Uses external plugins for multimedia.	• No external plugins needed.
• Has fewer semantic tags.	• provides semantic tags.
• No <canvas> element	• Supports <canvas>
• Limited web storage	• Supports local storage
• Complex DOCTYPE declaration.	• Simple <!DOCTYPE HTML>

CSS	CSS3
• Earlier version of CSS	• Advanced version of CSS.
• Basic styling features.	• Provides advanced styling features.
• No flexible layout systems.	• Supports flexbox and grid.
• Limited visual effects.	• Supports shadows, gradients, effects.
• No media queries.	• Supports media queries.
• Uses a single specification.	• Divides into different modules.

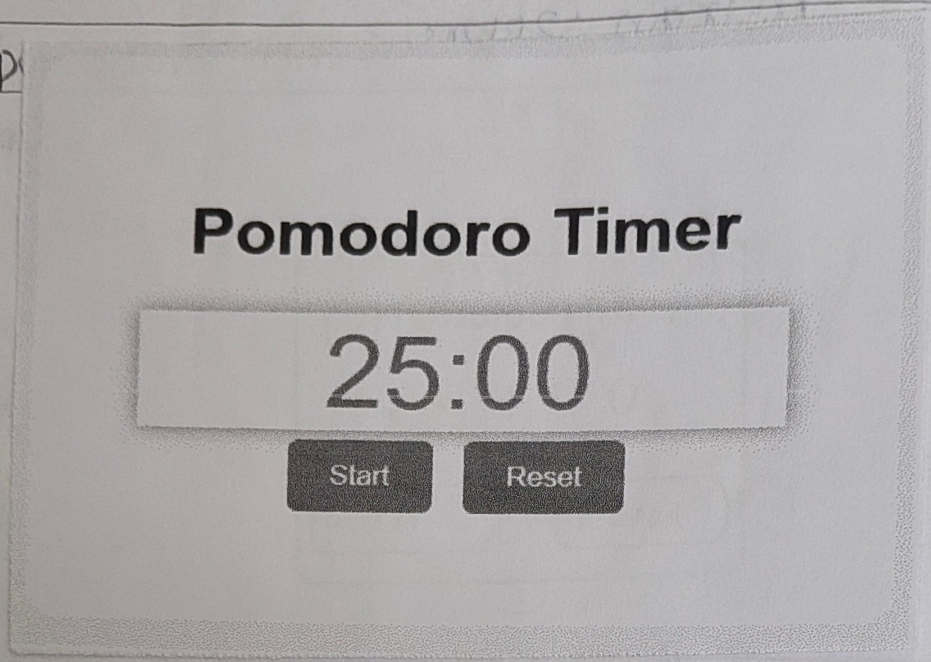
15/07/2026

Design a static webpage using HTML and CSS.

pomodoro Timer, using HTML, CSS

```
<!DOCTYPE html>
<html>
<head>
    <title>pomodoro Timer</title>
    <style>
        body{
            font-family: Arial;
            background: #f5f5f5;
            text-align: centre;
            padding: 50 px;
        }
        .timer{
            width: 300 px;
            margin: auto;
            padding: 30px;
            background: white;
            border-radius: 15 px;
            box-shadow : 0 0 10px gray;
        }
        h1{
            color: #e74c3c;
        }
```

```
time{
    font-size:50px;
    margin:20px;
}
button{
    padding: 10px 20px;
    margin: 5px;
    border: none;
    border-radius:5px;
    background: #e74c3c;
    color: white;
}
</style>
</head>
<body>
    <div class="timer">
        <h1>pomodoro Timer</h1>
        <div class="time"></button>
        <button>Reset</button>
    </div>
</body>
</html>
```


Pomodoro Timer
25:00
Start
Reset

i) Design an HTML code using audio tags.

Code:

```
<!DOCTYPE html>
<html>
<head>
    <title> AudioExample </title>
</head>
<body>
    <h2> my favourite song </h2>
    <audio controls>
        <source src= "songs.mp3" type=
        "audio/mpeg">
        your browser does not support the
        audio tag
    </audio>
</body>
</html>
```

O/p: Displays an audio player with play, pause

Steps:

- create a new html file.
- Add the basic HTML structure.
- Add heading using <h2> tag.
- Insert the <audio> tag with controls attribute.

- Specify the audio file using tag.
- Save and open the HTML file in a web
- browser.
- Play the audio using displayed controls.

5) Design a HTML code using video tags.

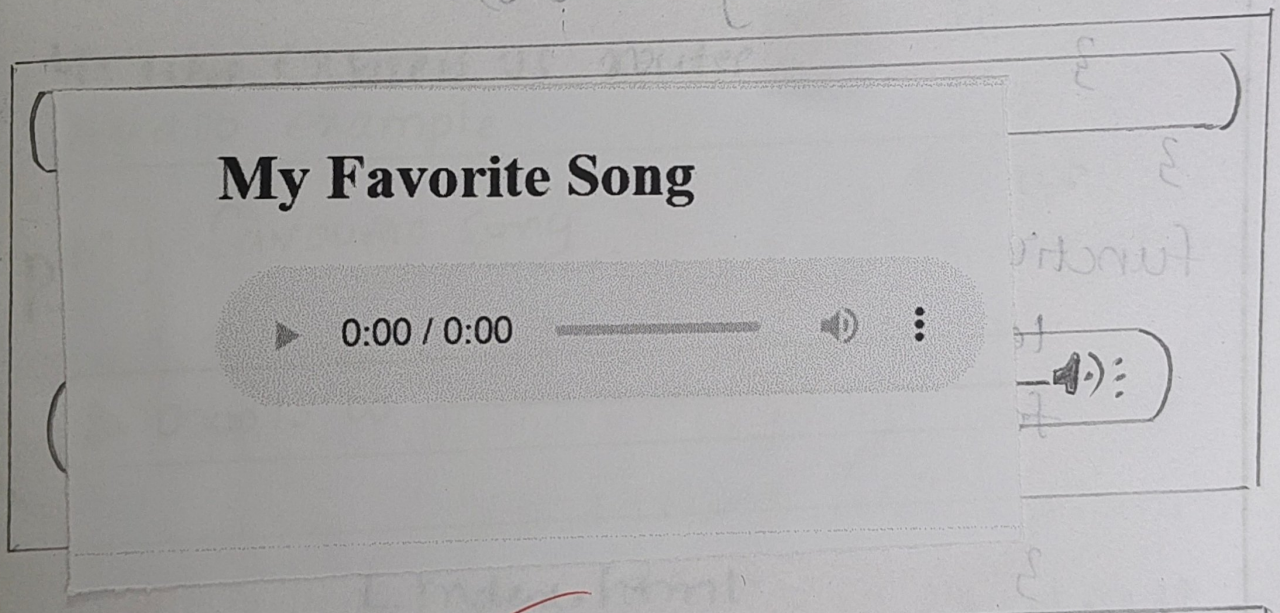
code:

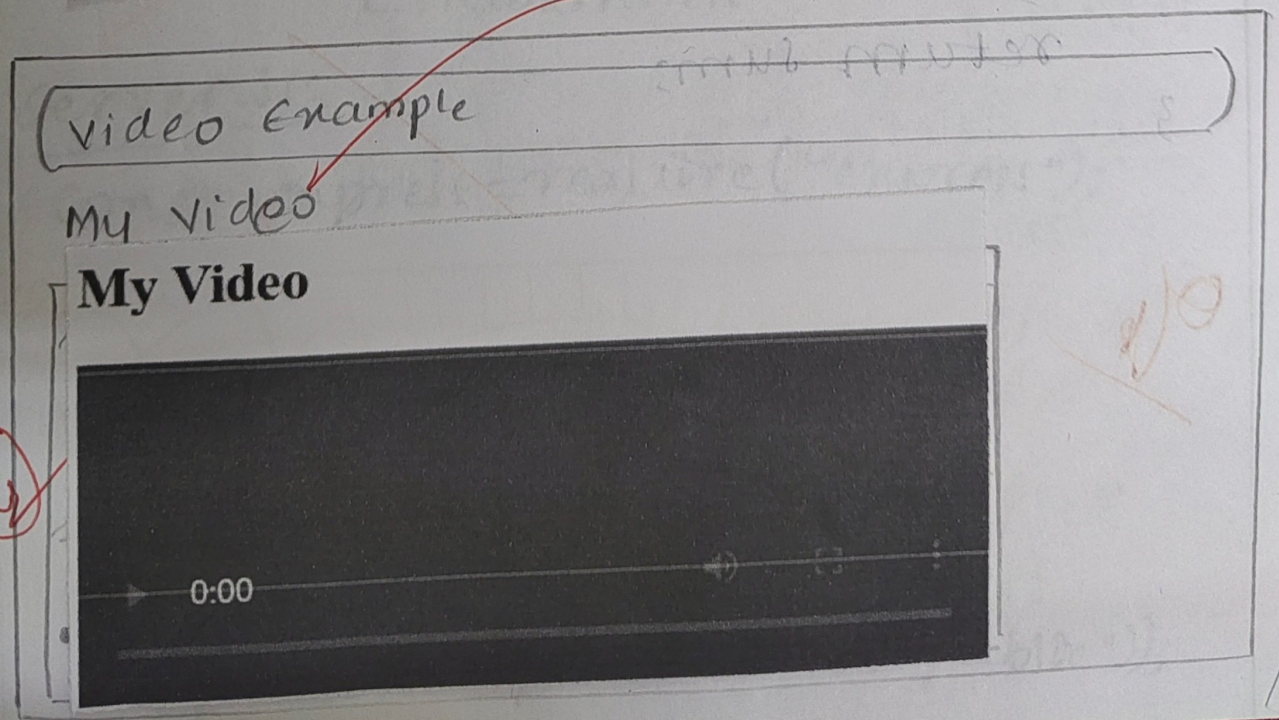
```
<!DOCTYPE html>
<html>
<head>
    <title> Video Example </title>
</head>
<body>
    <h2> My video</h2>
    <video width="400" controls>
        <source src="video.mp4" type
        "video/mp4">
    </video>
</body>
</html>
```

O/p: Displays a video player with play, pause.

steps:

- Creates a new HTML file.
- Add a basic HTML structure
- Add a heading using the <h2> tag
- Insert the <video> tag with control attribute.
- Specify the video file using <source> tag.
- Save the file and open it in web browser.
- Play the video using displayed controls.





Write a program using Javascript (arrays, functions).

Code:

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90];
function displayArray(arr){
    console.log("Array elements: ");
    for(let i=0; i<arr.length; i++){
        console.log(arr[i]);
    }
}
function findsum(arr){
    let sum=0;
    for(let i=0; i<arr.length; i++){
        sum+=arr[i];
    }
    return sum;
}
```

O/p:

Array elements:-

2 45 8 23 56 19 34 67 90

Sum: 276

Experiment - 06

06. Develop a mini application using express and NodeJS.

- Simple mini application using nodeJS + ExpressJS

Student management system:

1. Add a student
2. View all students
3. delete a student
4. Uses Express JS routes
5. Simple HTML interface

Project Structure:

```
student.appl
├── server.js
└── public
    └── index.html
```

server.js:

```
const express = require("express");
const app = express();
const PORT = 3000;
let students = [];
app.use(express.json());
app.use(express.static("public"));
```

```
app.get("/students", (req, res) => {
    res.json(students)
});

// Add
app.post("/students", (req, res) => {
    const student = {
        id: students.length + 1,
        name: req.body.name,
        course: req.body.course
    };
    students.push(student);
    res.json({ message: "student added
                        sucessfully",
               student: student })
});

// Delete
app.delete("/students/:id", (req, res) => {
    const id = parseInt(req.params.id);
    students = students.filter(student =>
        student.id !== id)
    res.json({
        message: "student deleted
```

```
});
// start server
app.listen(PORT, () => {
  console.log('server running at http://localhost:
  ${PORT}');
});
```

public/index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
</head>
<body>
    <h1>Student Management System</h1>
    <input type="text" id="name"
    placeholder="Enter student name">
    <input type="text" id="course"
    placeholder="Enter course">
    <button onclick="addStudent()">
        Add Student </button>
    <h2> Student List </h2>
```

```
<ul id = "studentlist"></ul>

<script>
  Function addstudent() {
    let name = document.getElement
          ById ("name").value;

    let course = document.getElementBy
          ("course").values

  fetch ("/students", {
          method: "post",
          headers: {
            "content-Type": "applicatio
                          json"},
        body: JSON.stringfy({
          name: name,
          course: course,
        3)
      3)
    .then (response => response.json
    .then (data => {
          alert (data.message);
document.getElementById ("name").
          value = "";
document.getElementById ("course").
          value = "";
    displaystudents();
```

```
});
}
function displayStudents(){
fetch("/students").then(response =>
                    response.json())
.then(students => {
  let list = document.getElementById(
                    "studentList");
  list.innerHTML = "";
students.forEach(student => {let li =
document.createElement("li");
 li.innerHTML = student.id + "-" +
                  student.name + "-" +
                  student.course + "<button
onclick="deleteStudent(" + student.id + ")
">Delete
</button>";
list.appendChild(li);
});
});
}
function deleteStudent(id){
  fetch("/students/" + id, {method:
  "DELETE"})
.then(response => response.json())
```

```
.then (data => {
        alert(data.message);
        displayStudents();
    });
}
displayStudents()
</script>
</body>
</html>
```

Output:

Student management system

Enter student name

Enter course

Add Student

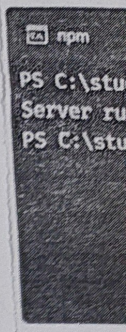
Student List:

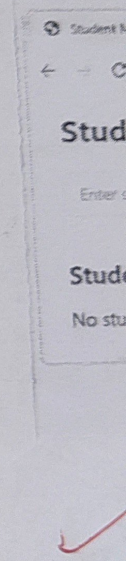
1. Dharani_CSM [delete]

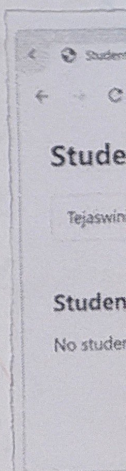
2. Nitish_CSE [delete]

- The Express.js is successfully started at http://localhost:3000
- The browser displays the student management system with fields.

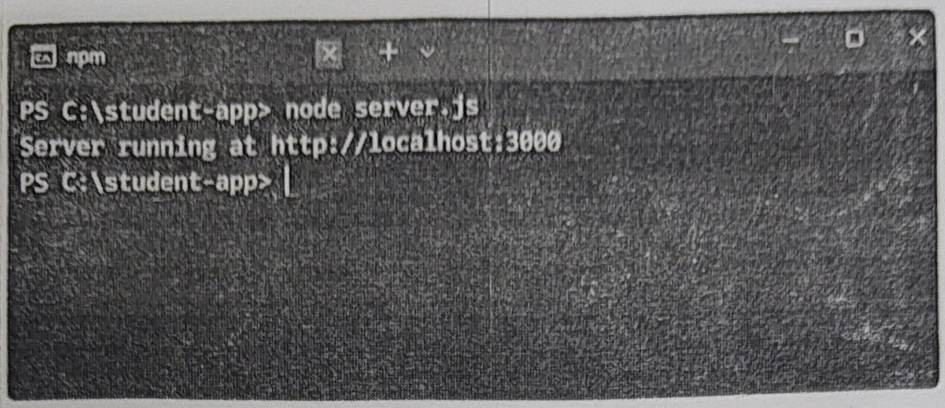
Output:



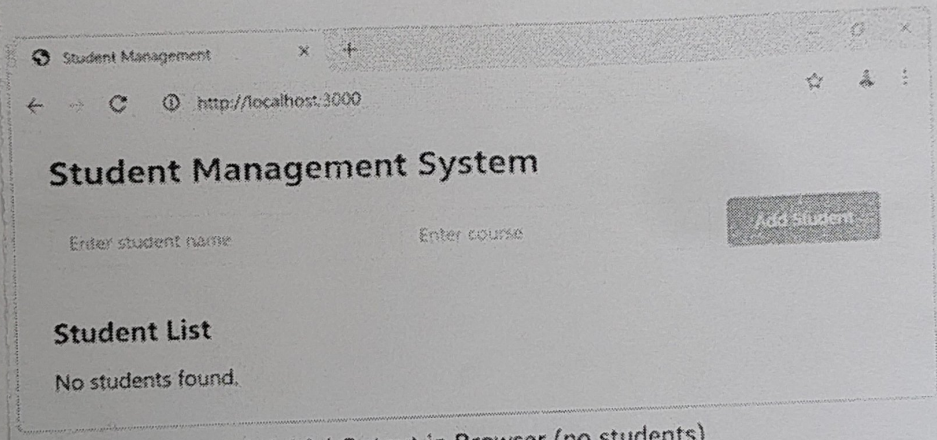




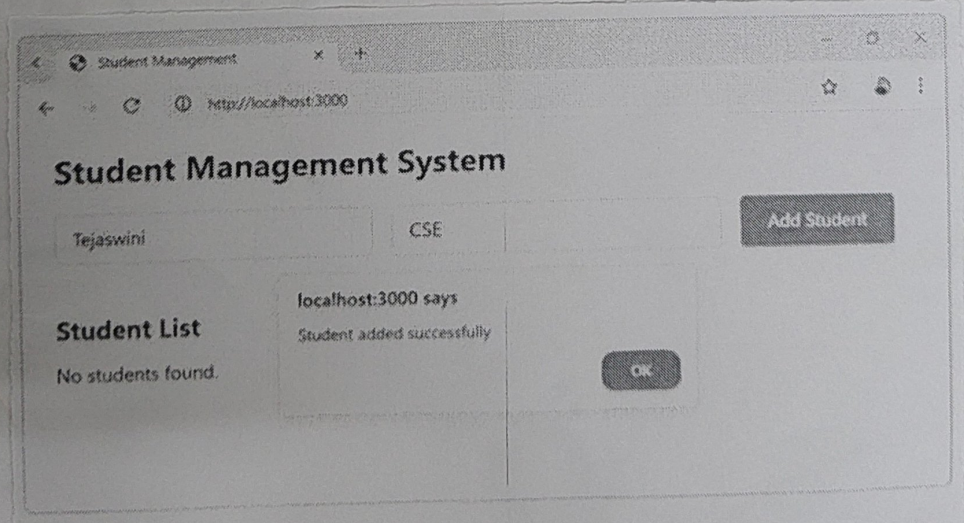
Output :



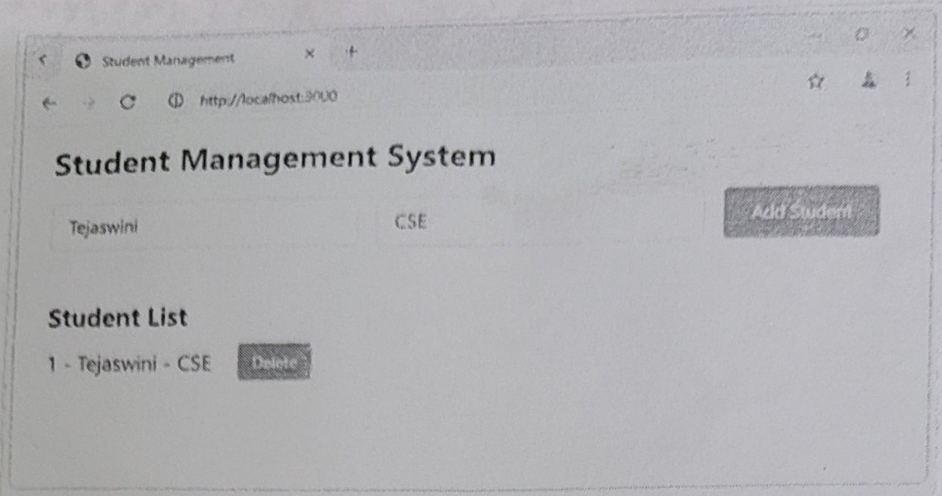
1. Terminal Output (after running `node server.js`)



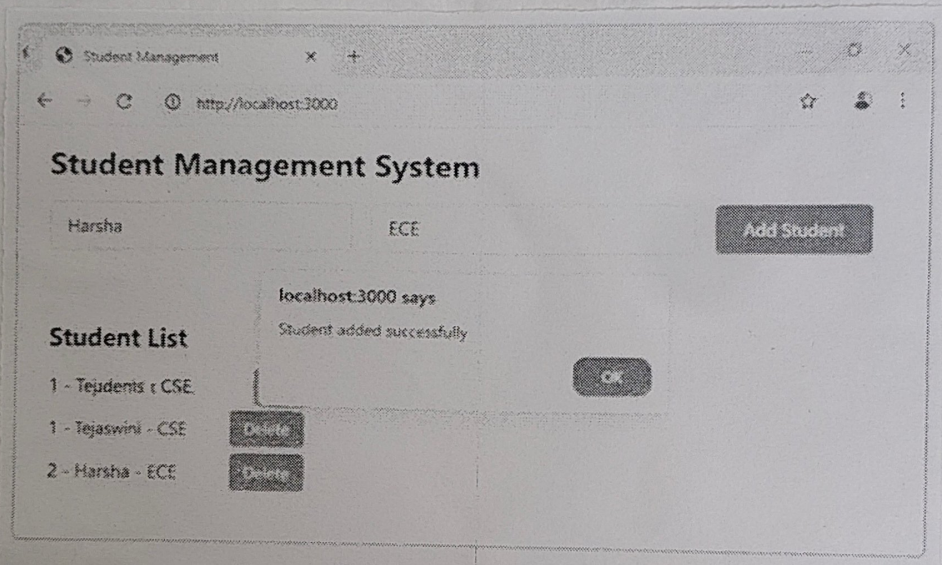
2. Initial Output in Browser (no students)



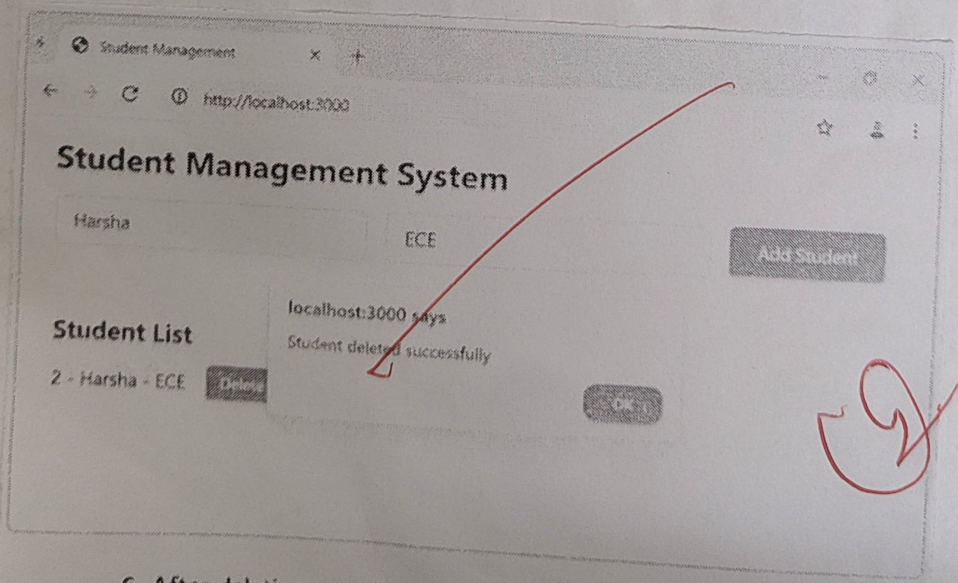
3. After adding a student (alert message)



4. Student list after adding one student



5. After adding another student (alert message)



6. After deleting a student (alert message and updated list)

Experiment - 04

4. Develop dynamic webpage using java script.

program:

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic webpage </title>
    <style>
        body{
            text-align: center;
            font-family: Arial;
            margin-top: 100px;
        }
    button{
        padding: 10px 20 px;
        font-size: 16px;
        cursor: pointer;
    }
    </style>
</head>
<body>
    <h1 id="heading"> Welcome to
        my webpage </h1>
    <p id="message"> click the button
        to change this webpage. </p>
    <button onclick="changepage()"> click me
    </button>
```

```
<script>
  function changepage(){
    document.getElementById("heading").
    innerHTML = "welcome to Javascript";
document.getElementById("heading").
    innerHTML = "The webpage has
    been changed dynamically";
document.body.style.backgroundcolor
    = "lightblue";
}
</script>
<body>
<html>
```

Output:

welcome to my webpage

click the button to change the webpage

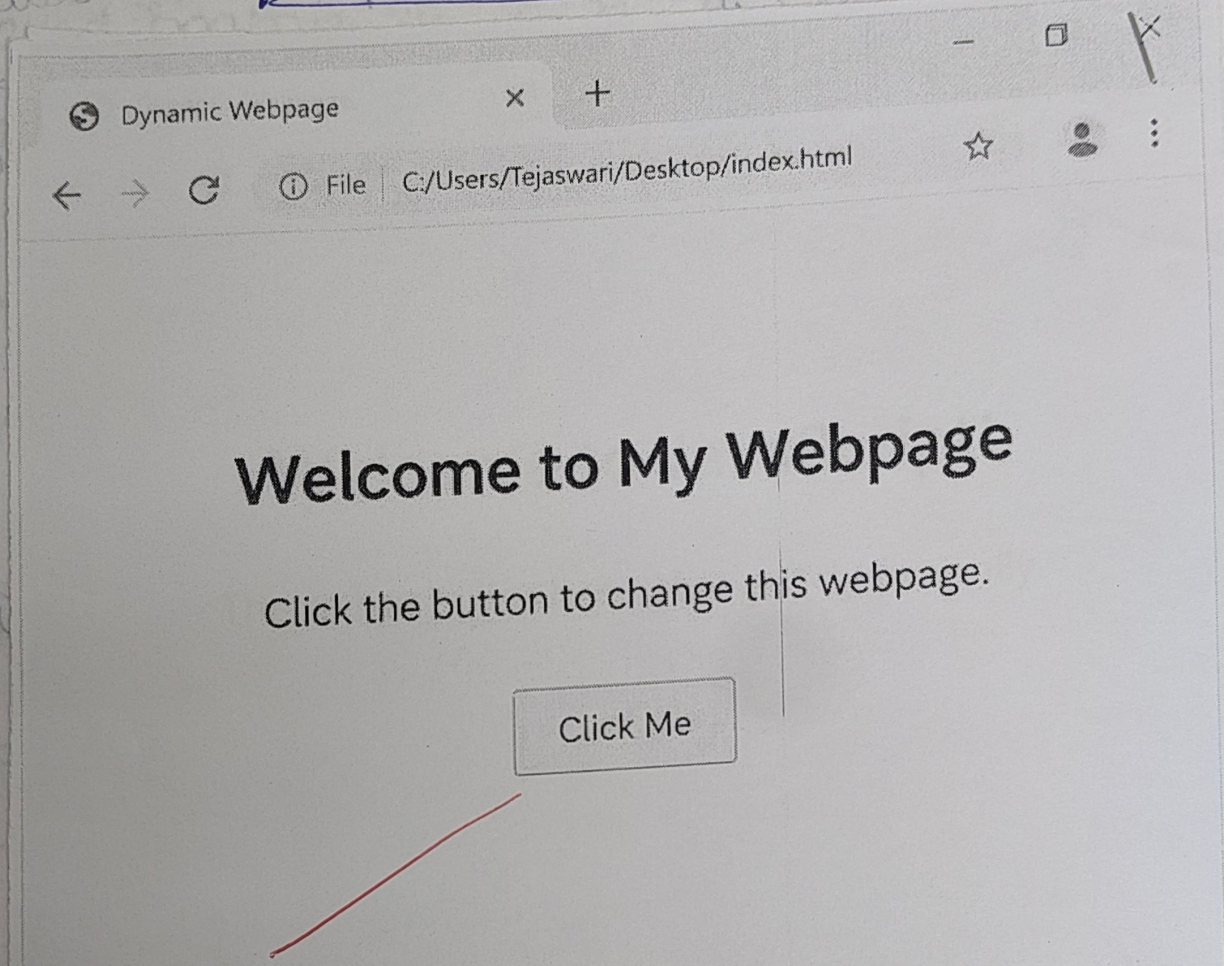
[clickme]

→ when the user clicks "clickme", the javascript function changepage() is called, then the heading dynamically changes and background also changes

- After clicking the button:
  welcome to javascript

The webpage has been changed dynamically

welcome



Before clicking the button

Experiment - 05

Implement modules in NodeJS.

. In Node-js, you can implement modules using two distinct systems: commonJS(JS), which is traditional default, and Es modules (ESM), the modern Javascript standard.

1. The commonJS method (default):

commonJS uses module.exports to share code and require() to import it. This system loads modules synchronously.

a. create the module (math.JS):

```
const add = (a,b) => a+b;
const subtract = (a,b) => a-b;
module.exports = {
    add,
    subtract
};
```

b. use the module (app.JS):

```
const math = require ('./math');
console.log(math.add (10,5));
Console.log(math.subtract (10,5));
```

O/p - 15
5

2. the Es modules method (modern standard):

Es modules use export to share code and import to use it. To make Node.js recognize Es modules, you must either use the .mjs file extension or add "type": "module" to your package.json file.

a) create the module (calculator.js):

```
export const multiply = (a,b) => a*b
export const divide = (a,b) => a/b

const calculatorName = "super-calc";
export default calculatorName;
```

b) use the module (main.js)

```
import calculatorName, {multiply, divide}
        from './calculator.js';

console.log(calculatorName);
console.log(multiply(4,2));
console.log(divide(4,2));
```

o/p- Supercalc
8
2

Node.js also comes equipped with dozens of built-in core modules (like fs for filesystem or path for file routing) that don't need installation.

We will load them by module name.

- // common JS style

```
const fs = require('fs');
```

// ES modules style

```
import fs from 'fs';
```

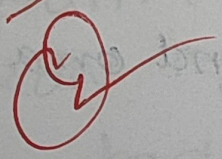
Here are some of the most commonly used ones:

1. fs = file system operations
2. http = HTTP server and client
3. path = file path utilities
4. os - Operating system utilities
5. events - Event handling
6. util - utility functions
7. stream - Stream handling
8. crypto - cryptographic functions
9. url - URL parsing
10. query string - URL

Eg: simple HTTP server

```
http.create server((req, res) => {
    res.writeHead(200, {'content-type':
                        'text/html'});
    res.end("Hello world!");
}).listen(8080);
```

O/p: Hello world!



Experiment-07

07. Create a functional component and class component using react.

Aim:- To develop a react application using functional components and class components to display student details, manage state, handle button click and dynamically update data and on a webpage

functional component:

A functional component is a javascript function that returns JSX to display the user interface. It uses React Hooks such as useState to manage the state and update the webpage dynamically.

program:

```
import React, {useState} from
'react';
function StudentFunctional(){
    const [name, setName]=useState("Dha
    const [course, setCourse]=useState("CSM
    const [count, setCount]= useState(0);
    const changeName =() => {
```

```
};};
const changecourse=()=>{
    setcourse("CSD");
};
const incrementCount=()=>{
    setCount(count+1);
};
return(
<div style={{
    border:"2px solid blue",
    padding:"20px",
    margin:"20px",
    borderRadius:"10px",
    textAlign:"center"
}}>
<h1>Student Details</h1>
<h2>functional component</h2>
<p><b>Name:</b>{name}</p>
<p><b>course:</b>{course}</p>
<p><b>Button clicks:</b>{count}</p>
<button onClick={changeName}>
    changeName</button>
<button onClick={change course}>change
    course</button>
```

<button onClick={...}>

</button>

);

Output: When the user clicks the button the name, course and count change dynamically without reloading the webpage

class component:

A class component is an ES6 Javascript class that extends React.Component. It uses the render() method to return JSX and manages state using this.state and this.setState().

program:

```
import React, {Component} from
'react';
class Studentclass extends Component{
    constructor(props){
        super(props);
        this.state={
            name: "Dharani"
            course: "computer science"
            count: 0
        };
    }
ChangeName =()=>{
    this.setState({
        name: "Dhar"
        });
ChangeCourse =()=>{
    this.setState({
        course: "CSM"
        });
```

```
incrementCount = () => {
  this.setState(prevState => {
    count = prevState.count + 1
  }));
};
render() {
  return (
    <div style={{
      border: "2px solid green",
      padding: "20px",
      margin: "20px",
      borderRadius: "10px",
      textAlign: "center"
    }}>
    <h1>Student Details</h1>
    <h2>Class Component</h2>
<p><b>Name:</b> {this.state.name}</p>
<p><b>Course:</b> {this.state.course}</p>
<p><b>Button clicks:</b> {this.state.count}</p>
<button onClick={this.changeName}>
  changeName</button>
<button onClick={this.changeCourse}>
  changeCourse</button>
```

t state(prevState) => {

ount = prevState.count + 1

<button onClick={this.incrementCount}>
Increment</button>

Output:

st:3000

React Components

Student Details

Functional Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Student Details

Class Component

Name: Tejaswari

Course: CSE

Button Clicks: 0

Change Name | Change Course | Increment

Experiment-08

08. Implement http methods using JSON Server.

Aim: To implement HTTP methods such as GET, POST, PUT, PATCH and DELETE using JSON server to perform CRUD (Create Read, Update and Delete) operations on student records.

Theory: JSON server is a tool used to create a fake REST API using a JSON file. It allows us to perform HTTP operations without developing a backend server from scratch.

HTTP methods are used to communicate between a client and a server.

- GET - Retrieves data from the server.
- POST - Adds new data to the server.
- PUT - Replaces an existing record.
- PATCH - updates selected fields of a record
- DELETE - Removes a record from the server

a. Create a JSON Server:

Step 1: Initialize the project

Open the ... because the following commands:

```
mkdir JSONServer
cd JSONServer
npm init -y
npm install json server.
```

Step-2: create db.json

Create a file named db.json and add the following student data.

```
{
  "student"=[
    {
      "id" = "1",
      "name" = "Dharani"
      "course" = "CSM",
      "marks" = "90".
    },
    {
      "id" = "3",
      "name" = "Nithish",
      "course" = "IT",
      "marks" = 88
    }
```

Step 3: Start the JSON server

For the current JSON server version, execute

npx json-server db.json

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2. Implementation of HTTP methods:

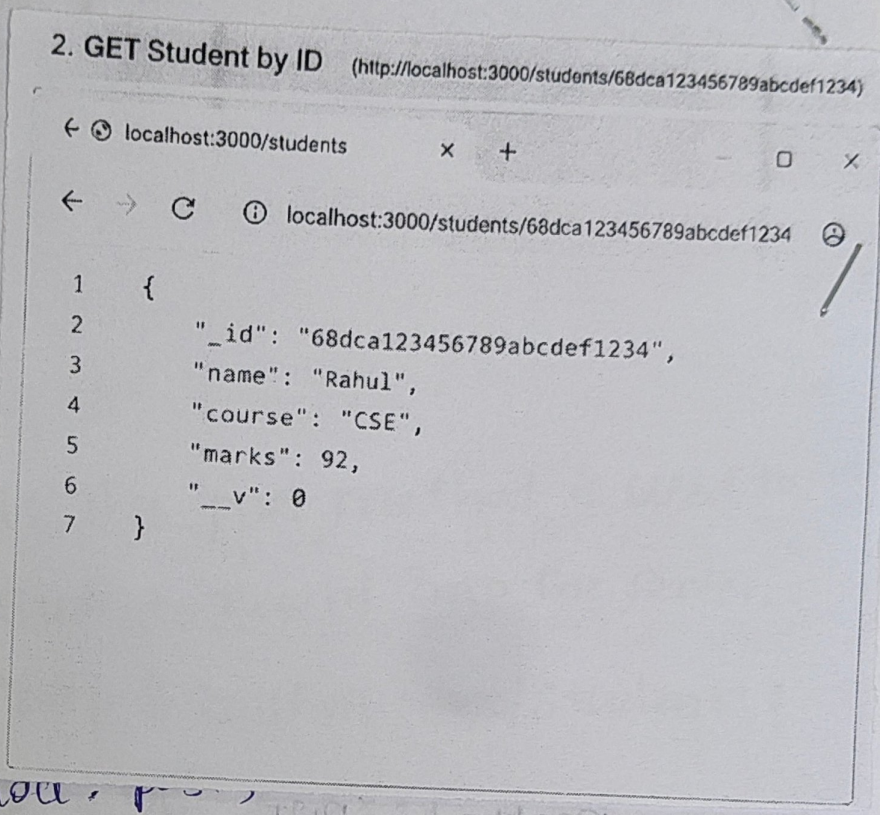
a. GET method: The GET method is used to retrieve student records from JSON server

code:

```
fetch('http://localhost:3000/students')
.then(response => response.json());
.then(data => {
        console.log(data);
});
.catch(error => {
```

```
    console.log(error);
});
```

Output:



B. POST Method

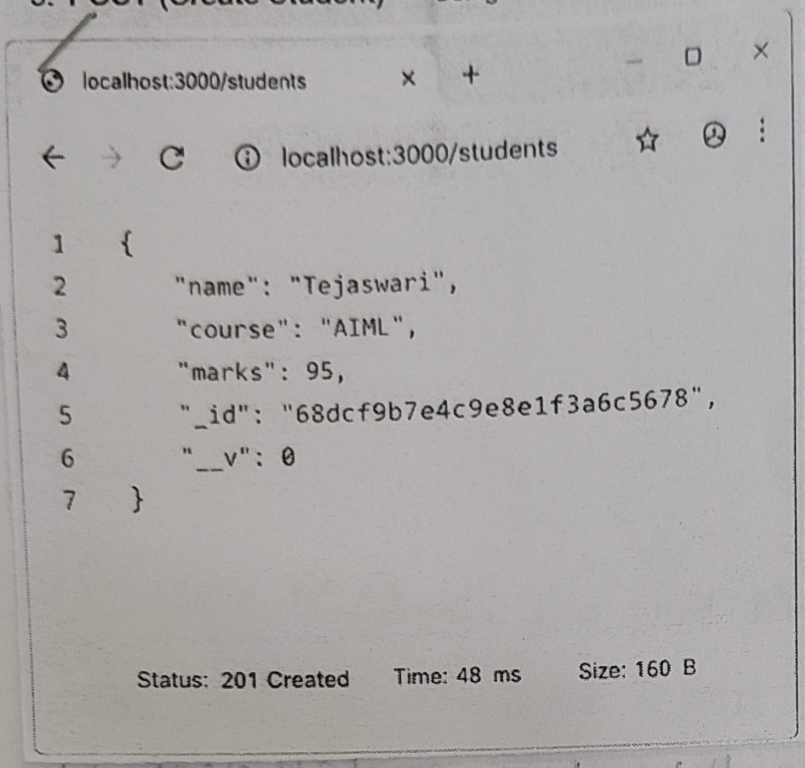
Insert a new

Code: fetch(

```
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({
      name: 'Rahul',
      course: 'CSE',
      marks: 92
    })
  })
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

```
• Catch (error =>{
    console.log(error);
});
```

3. POST (Create Student) – Using Postman

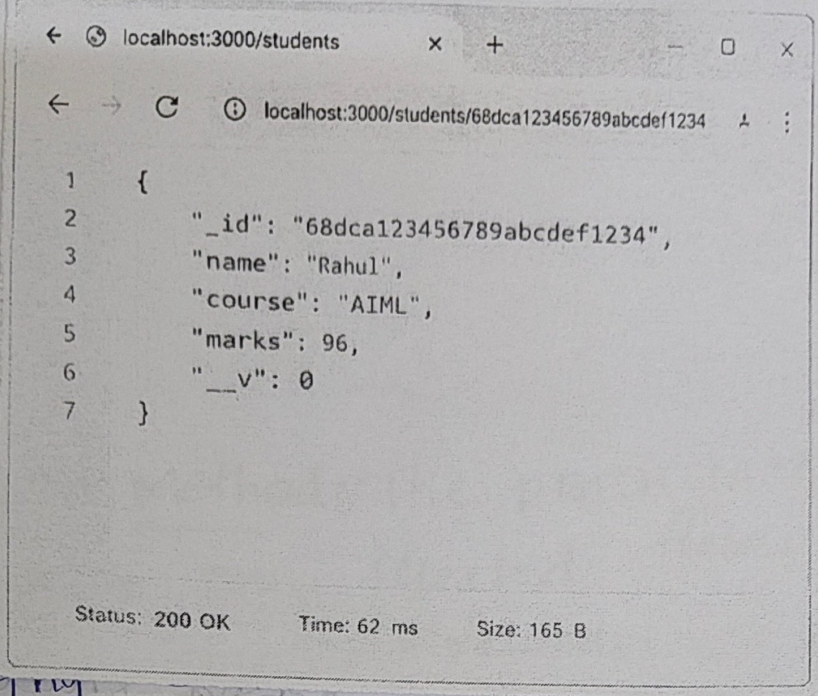


ethod is used to replace
with updated data.

code:
```
fetch('http://localhost:3000/students/2',{
    method: 'PUT',
    headers:{
      'content-Type': 'application/json'
    },
    body: JSON.stringify({
        name: 'Dharani',
        Course: 'AIML',
    •then (data =>{
            console.log(data);
    })
    • Catch (error =>{
            console.log(error);
    });
```

Outp

4. PUT (Update Student) – Using Postman



D.PA

od is used

ot an

to u

existing

code: fetch('http://localhost:3000/students/2',{

method: 'PATCH',

headers: {

'Content-Type': 'application/json'

},

body: JSON.stringify({

marks: 96

})

})

.then(response => response.json())

.then(data => {

console.log(data)

})

.catch(error => {

console.log(error);

});

6. GET All Students (After Deletion)

localhost:3000/students

localhost:3000/students

1 []
2

Status: 200 OK Time: 28 ms Size: 2 B

E. DELET

used to remove an existing student record from the server.

code: fetch("http://localhost:3000/students/

method: DELETE

3)

.then (responce =>{

if (response.ok){

console.log ("student deleted successfully");

}

3)

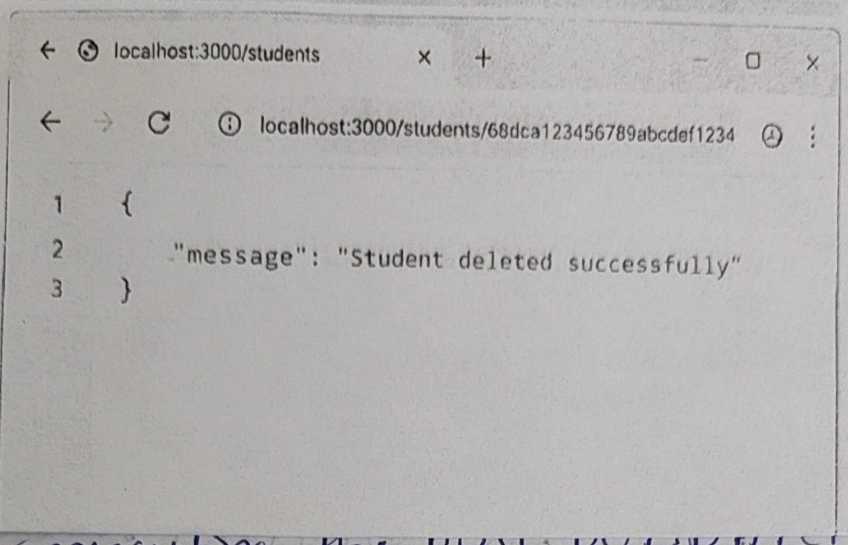
.catch (error =>{

console.log(error)

3);

Outp

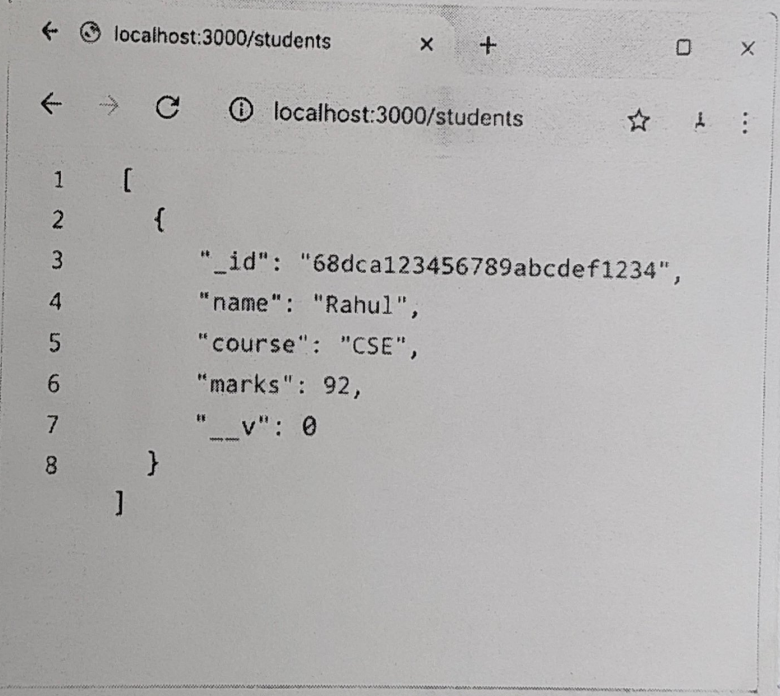
5. DELETE (Remove Student) – Using Postman



• After executing the post, put, patch, and DELETE operations in sequence, the remaining student will be

Output:

1. GET All Students (http://localhost:3000/students)



Thus, HTTP methods GET, POST, PUT, PATCH, and DELETE were successfully implemented using JSON Server. The CRUD operations were performed on student records. The data was retrieved, inserted, updated and deleted using HTTP requests.

Experiment - 09

09. Develop CRUD operations using Mongo

Aim: To develop a web application using Node.js, express.js, and MongoDB to perform CRUD operations (create, read, update, delete) on student records.

Theory: MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in a database.

Mongoose is an object Data Modeling (ODM) library for MongoDB and Node.js. It helps define schemas, interact with collections and perform database operations.

1. **Installation and project setup:**

```
mkdir MongoDB-CRUD
cd MongoDB-CRUD
npm init -y
npm install express mongoose
```

program: Servers

```
const express. require ('express')
const mongoose. require ('mongoose');
const app = express();
const port = 3000;
app.use(express.json());
mongoose.connect('mongodb://127.0.0.1:
  27017/college)
.then() => console.log("MongoDB Connected"))
.(catch(err=>) console.log(err));
//create student
const studentSchema=new
mongoose.schema({
    name: {
      type: string
      required: true
    },
    course{
      type: string,
      required: true
    },
  marks:{
      type: number
      required: true
  });
```

```
//create student model
const student =
    mongoose.model('student', studentSch
//CREATE: Add
app.post('/students', async(req, res)=>{
  try{
  const student = new student(req.body
  const savedstudent=await student.save
  res.status(201).json(savedstudent);
  }catch(error){
     res.status(400).json({message:
           error:message});
  }
});
//READ: Get all students
app.get('/students', async(req, res)=>{
  try{
   const students=await
student.find();
  res.status(500).json({message:error.
                    message});
  }
});
```

```
//READ : Get student by ID
app.get('/students/:id', async (req, res) => {
  try{
    const student = await
    Student.findById(req.params.id);
    if(!student){
      return res.status(404).json({
        message: "student not found"
      });
    }
  res.status(200).json(student);
  }catch(error){
    res.status(400).json({message:
      error.message});
  }
});
//UPDATE
app.put('/students/:id', async(req,res)=>{
  try{
  const student=await
Student.findByIdandUpdate(
  req.params.id,
  req.body,
{new: true, runValidators: true}
);
```

```
if(!student){
    return res.status(404).json({
        message: "student not found"
    });
}
res.status(200).json(student);
}catch(error){
    res.status(400).json({message:
        error.message});
}
});

//DELETE
app.delete('/students/:id', async(req,res)
    try{
    const student = await Student.
        findByIdAndDelete(
        req.params.id
    );
if(!student){
    return res.status(404).json({
        message: "student not found"
    });
}
```

message: "Student deleted successfully"

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
```

1. Terminal Output

localhost:3000

localhost:3000

MongoDB CRUD API

Server is running...

2. Chrome Browser (Home Page)

POST http://localhost:3000/students Send

Params Authorization Headers (9) Body Scripts Settings

raw JSON

```
{
    "name": "Tejaswari",
    "course": "CSE",
    "marks": 85
}
```

Body Cookies Headers (7) Test Results

201 Created 188 ms 268 B

```
{
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "name": "Tejaswari",
    "course": "CSE",
    "marks": 85,
    "__v": 0
}
```

3. Create (POST) – Add Student

GET http://localhost:3000/students Send

Params Authorization Headers (7) Body Scripts Settings

raw JSON

200 OK 32 ms 512 B

```
[
    {
        "_id": "68da1f2e4c9e8e1f3a6c1234",
        "name": "Tejaswari",
        "course": "CSE",
        "marks": 85,
        "__v": 0
    },
    {
        "_id": "68da1f3a4c9e8e1f3a6c5678",
        "name": "Rohit",
        "course": "AIML",
        "marks": 92,
        "__v": 0
    }
]
```

4. Read (GET) – Get All Students

GET http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (7) Body Scripts Settings

raw JSON

200 OK 28 ms 246 B

```
{
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "name": "Tejaswari",
    "course": "CSE",
    "marks": 85,
    "__v": 0
}
```

5. Read (GET) – Get Student by ID

PUT http://localhost:3000/students/68da1f2e4c9e8e1f3a6c1234 Send

Params Authorization Headers (9) Body Scripts Settings

raw JSON

```
{
    "name": "Tejaswari",
    "course": "AIML",
    "marks": 95
}
```

Body Cookies Headers (7) Test Results

200 OK 24 ms 268 B

```
{
    "_id": "68da1f2e4c9e8e1f3a6c1234",
    "coure": "Tejaswari",
    "course": "AIML",
    "marks": 95,
    "__v": 0
}
```

6. Update (PUT) – Update Student

DELETE http://localhost:3000/students/68da1f3a4c9e8e1f3a6c5678 Send

Params Authorization Headers (9) Body Scripts Settings

Body Pretty Raw Preview Visualize JSON

200 OK 20 ms 210 B

```
{
    "message": "Student deleted successfully"
}
```

7. Delete (DELETE) – Remove Student

MongoDB Compass

Local

localhost:27017

college

students

admin

config

local

Open MongoDB Shell Refresh

college.students

Documents Aggregations Schema Indexes

Filter

ADD DATA

_id	name	course	marks	__v
68da1f2e4c9e8e1f3a6c1234	Tejaswari	AIML	95	0
68da1f3a4c9e8e1f3a6c5678	Rohit	AIML	92	0

8. MongoDB Compass – Database View